JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DateFormatSymbols

java.lang.Object
    java.text.DateFormatSymbols

All Implemented Interfaces:

`Serializable`, `Cloneable`

```
public class DateFormatSymbols
extends Object
implements Serializable, Cloneable
```

`DateFormatSymbols` is a public class for encapsulating localizable date-time formatting data, such as the names of the months, the names of the days of the week, and the time zone data. `SimpleDateFormat` uses `DateFormatSymbols` to encapsulate this information.

Typically you shouldn't use `DateFormatSymbols` directly. Rather, you are encouraged to create a date-time formatter with the `DateFormat` class's factory methods: `getTimeInstance`, `getDateInstance`, or `getDateTimeInstance`. These methods automatically create a `DateFormatSymbols` for the formatter so that you don't have to. After the formatter is created, you may modify its format pattern using the `setPattern` method. For more information about creating formatters using `DateFormat`'s factory methods, see `DateFormat`.

If you decide to create a date-time formatter with a specific format pattern for a specific locale, you can do so with:

```
new SimpleDateFormat(aPattern, DateFormatSymbols.getInstance(aLocale));
```

If the locale contains "rg" (region override) Unicode extension, the symbols are overridden for the designated region.

`DateFormatSymbols` objects are cloneable. When you obtain a `DateFormatSymbols` object, feel free to modify the date-time formatting data. For instance, you can replace the localized date-time format pattern characters with the ones that you feel easy to remember. Or you can change the representative cities to your favorite ones.

New `DateFormatSymbols` subclasses may be added to support `SimpleDateFormat` for date-time formatting for additional locales.

Since:

1.1

See Also:

DateFormat, SimpleDateFormat, SimpleTimeZone, Serialized Form

Constructor Summary

Constructors

Constructor	Description
`DateFormatSymbols()`	Construct a DateFormatSymbols object by loading format data from resources for the default `FORMAT` locale.
`DateFormatSymbols(Locale locale)`	Construct a DateFormatSymbols object by loading format data from resources for the given locale.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`Object`	`clone()`	Overrides Cloneable
`boolean`	`equals(Object obj)`	Compares the specified object with this `DateFormatSymbols` for equality.
`String[]`	`getAmPmStrings()`	Gets ampm strings.
`static Locale[]`	`getAvailableLocales()`	Returns an array of all locales for which the `getInstance` methods of this class can return localized instances.
`String[]`	`getEras()`	Gets era strings.
`static final DateFormatSymbols`	`getInstance()`	Gets the `DateFormatSymbols` instance for the default locale.

static final DateFormatSymbols	**getInstance**(Locale locale)	Gets the DateFormatSymbols instance for the specified locale.
String	**getLocalPatternChars**()	Gets localized date-time pattern characters.
String[]	**getMonths**()	Gets month strings.
String[]	**getShortMonths**()	Gets short month strings.
String[]	**getShortWeekdays**()	Gets short weekday strings.
String[]	**getWeekdays**()	Gets weekday strings.
String[][]	**getZoneStrings**()	Gets time zone strings.
int	**hashCode**()	Returns the hash code for this DateFormatSymbols.
void	**setAmPmStrings**(String[] newAmpms)	Sets ampm strings.
void	**setEras**(String[] newEras)	Sets era strings.
void	**setLocalPatternChars**(String newLocalPatternC	Sets localized date-time pattern characters.
void	**setMonths**(String[] newMonths)	Sets month strings.
void	**setShortMonths**(String[] newShortMonths	Sets short month strings.
void	**setShortWeekdays**(String[] newShortWeekda	Sets short weekday strings.
void	**setWeekdays**(String[] newWeekdays)	Sets weekday strings.
void	**setZoneStrings**(String[][] newZoneStrings)	Sets time zone strings.

Methods declared in class Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Constructor Details

DateFormatSymbols

```
public DateFormatSymbols()
```

Construct a DateFormatSymbols object by loading format data from resources for the default FORMAT locale. This constructor can only construct instances for the locales supported by the Java runtime environment, not for those supported by installed DateFormatSymbolsProvider implementations. For full locale coverage, use the getInstance method.

This is equivalent to calling DateFormatSymbols(Locale.getDefault(Locale.Category.FORMAT)).

Throws:

MissingResourceException - if the resources for the default locale cannot be found or cannot be loaded.

See Also:

getInstance(), Locale.getDefault(java.util.Locale.Category), Locale.Category.FORMAT

DateFormatSymbols

```
public DateFormatSymbols(Locale locale)
```

Construct a DateFormatSymbols object by loading format data from resources for the given locale. This constructor can only construct instances for the locales supported by the Java runtime environment, not for those supported by installed DateFormatSymbolsProvider implementations. For full locale coverage, use the getInstance method.

Parameters:

locale - the desired locale

Throws:

MissingResourceException - if the resources for the specified locale cannot be found or cannot be loaded.

NullPointerException - if locale is null

See Also:

`getInstance(Locale)`

Method Details

getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all locales for which the `getInstance` methods of this class can return localized instances. The returned array represents the union of locales supported by the Java runtime and by installed `DateFormatSymbolsProvider` implementations. At a minimum, the returned array must contain a `Locale` instance equal to `Locale.ROOT` and a `Locale` instance equal to `Locale.US`.

Returns:

An array of locales for which localized `DateFormatSymbols` instances are available.

Since:

1.6

getInstance

```
public static final DateFormatSymbols getInstance()
```

Gets the `DateFormatSymbols` instance for the default locale. This method provides access to `DateFormatSymbols` instances for locales supported by the Java runtime itself as well as for those supported by installed `DateFormatSymbolsProvider` implementations.

This is equivalent to calling `getInstance(Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

a `DateFormatSymbols` instance.

Since:

1.6

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getInstance

```
public static final DateFormatSymbols getInstance(Locale locale)
```

Gets the `DateFormatSymbols` instance for the specified locale. This method provides access to `DateFormatSymbols` instances for locales supported by the Java runtime itself as well as for those supported by installed `DateFormatSymbolsProvider` implementations.

Parameters:

`locale` - the given locale.

Returns:

a `DateFormatSymbols` instance.

Throws:

`NullPointerException` - if `locale` is null

Since:

1.6

getEras

```
public String[] getEras()
```

Gets era strings. For example: "AD" and "BC".

Returns:

the era strings.

setEras

```
public void setEras(String[] newEras)
```

Sets era strings. For example: "AD" and "BC".

Parameters:

`newEras` - the new era strings.

getMonths

```
public String[] getMonths()
```

Gets month strings. For example: "January", "February", etc. An array with either 12 or 13 elements will be returned depending on whether or not `Calendar.UNDECIMBER` is supported. Use `Calendar.JANUARY`, `Calendar.FEBRUARY`, etc. to index the result array.

If the language requires different forms for formatting and stand-alone usages, this method returns month names in the formatting form. For example, the preferred month name for January in the Czech language is *ledna* in the formatting form, while it is *leden* in the stand-alone form. This method returns `"ledna"` in this case. Refer to the Calendar Elements in the

Unicode Locale Data Markup Language (LDML) specification for more details.

Implementation Requirements:

This method returns 13 elements since `Calendar.UNDECIMBER` is supported.

Returns:

the month strings.

External Specifications

Unicode Locale Data Markup Language (LDML)

setMonths

```
public void setMonths(String[] newMonths)
```

Sets month strings. For example: "January", "February", etc.

Parameters:

`newMonths` - the new month strings. The array should be indexed by `Calendar.JANUARY`, `Calendar.FEBRUARY`, etc.

getShortMonths

```
public String[] getShortMonths()
```

Gets short month strings. For example: "Jan", "Feb", etc. An array with either 12 or 13 elements will be returned depending on whether or not `Calendar.UNDECIMBER` is supported. Use `Calendar.JANUARY`, `Calendar.FEBRUARY`, etc. to index the result array.

If the language requires different forms for formatting and stand-alone usages, this method returns short month names in the formatting form. For example, the preferred abbreviation for January in the Catalan language is *de gen.* in the formatting form, while it is *gen.* in the stand-alone form. This method returns `"de gen."` in this case. Refer to the Calendar Elements in the Unicode Locale Data Markup Language (LDML) specification for more details.

Implementation Requirements:

This method returns 13 elements since `Calendar.UNDECIMBER` is supported.

Returns:

the short month strings.

External Specifications

Unicode Locale Data Markup Language (LDML)

setShortMonths

```
public void setShortMonths(String[] newShortMonths)
```

Sets short month strings. For example: "Jan", "Feb", etc.

Parameters:

`newShortMonths` - the new short month strings. The array should be indexed by `Calendar.JANUARY`, `Calendar.FEBRUARY`, etc.

getWeekdays

```
public String[] getWeekdays()
```

Gets weekday strings. For example: "Sunday", "Monday", etc.

Returns:

the weekday strings. Use `Calendar.SUNDAY`, `Calendar.MONDAY`, etc. to index the result array.

setWeekdays

```
public void setWeekdays(String[] newWeekdays)
```

Sets weekday strings. For example: "Sunday", "Monday", etc.

Parameters:

`newWeekdays` - the new weekday strings. The array should be indexed by `Calendar.SUNDAY`, `Calendar.MONDAY`, etc.

getShortWeekdays

```
public String[] getShortWeekdays()
```

Gets short weekday strings. For example: "Sun", "Mon", etc.

Returns:

the short weekday strings. Use `Calendar.SUNDAY`, `Calendar.MONDAY`, etc. to index the result array.

setShortWeekdays

```
public void setShortWeekdays(String[] newShortWeekdays)
```

Sets short weekday strings. For example: "Sun", "Mon", etc.

Parameters:

`newShortWeekdays` - the new short weekday strings. The array should be indexed by `Calendar.SUNDAY`, `Calendar.MONDAY`, etc.

getAmPmStrings

```
public String[] getAmPmStrings()
```

Gets ampm strings. For example: "AM" and "PM".

Returns:

the ampm strings.

setAmPmStrings

```
public void setAmPmStrings(String[] newAmpms)
```

Sets ampm strings. For example: "AM" and "PM".

Parameters:

`newAmpms` - the new ampm strings.

getZoneStrings

```
public String[][] getZoneStrings()
```

Gets time zone strings. Use of this method is discouraged; use `TimeZone.getDisplayName()` instead.

The value returned is a two-dimensional array of strings of size *n* by *m*, where *m* is at least 5. Each of the *n* rows is an entry containing the localized names for a single `TimeZone`. Each such row contains (with `i` ranging from 0..*n*-1):

- `zoneStrings[i][0]` - time zone ID
- `zoneStrings[i][1]` - long name of zone in standard time
- `zoneStrings[i][2]` - short name of zone in standard time
- `zoneStrings[i][3]` - long name of zone in daylight saving time
- `zoneStrings[i][4]` - short name of zone in daylight saving time

The zone ID is *not* localized; it's one of the valid IDs of the `TimeZone` class that are not custom IDs. All other entries are localized names. If a zone does not implement daylight saving time, the daylight saving time names should not be used.

If `setZoneStrings` has been called on this `DateFormatSymbols` instance, then the strings provided by that call are returned. Otherwise, the returned array contains names provided by the Java runtime and by installed `TimeZoneNameProvider` implementations.

Returns:

the time zone strings.

See Also:

`setZoneStrings(String[][])`

setZoneStrings

```
public void setZoneStrings(String[][] newZoneStrings)
```

Sets time zone strings. The argument must be a two-dimensional array of strings of size *n* by *m*, where *m* is at least 5. Each of the *n* rows is an entry containing the localized names for a single `TimeZone`. Each such row contains (with `i` ranging from 0..*n*-1):

- `zoneStrings[i][0]` - time zone ID
- `zoneStrings[i][1]` - long name of zone in standard time
- `zoneStrings[i][2]` - short name of zone in standard time
- `zoneStrings[i][3]` - long name of zone in daylight saving time
- `zoneStrings[i][4]` - short name of zone in daylight saving time

The zone ID is *not* localized; it's one of the valid IDs of the `TimeZone` class that are not custom IDs. All other entries are localized names.

Parameters:

`newZoneStrings` - the new time zone strings.

Throws:

`IllegalArgumentException` - if the length of any row in `newZoneStrings` is less than 5

`NullPointerException` - if `newZoneStrings` is null

See Also:

`getZoneStrings()`

getLocalPatternChars

```
public String getLocalPatternChars()
```

Gets localized date-time pattern characters. For example: 'u', 't', etc.

Returns:

the localized date-time pattern characters.

setLocalPatternChars

```
public void setLocalPatternChars(String newLocalPatternChars)
```

Sets localized date-time pattern characters. For example: 'u', 't', etc.

Parameters:

`newLocalPatternChars` - the new localized date-time pattern characters.

clone

```
public Object clone()
```

Overrides Cloneable

Overrides:

clone in class Object

Returns:

a clone of this instance.

See Also:

Cloneable

hashCode

```
public int hashCode()
```

Returns the hash code for this DateFormatSymbols.

Overrides:

hashCode in class Object

Implementation Requirements:

Non-transient instance fields of this class are used to calculate a hash code value which adheres to the contract defined in Objects.hashCode(Object).

Returns:

the hash code for this DateFormatSymbols

See Also:

Object.hashCode()

equals

```
public boolean equals(Object obj)
```

Compares the specified object with this DateFormatSymbols for equality. Returns true if the object is also a DateFormatSymbols and the two DateFormatSymbols objects represent the same date-time formatting data.

Overrides:

equals in class Object

Implementation Requirements:

This method performs an equality check with a notion of class identity based on getClass(), rather than instanceof. Therefore, in the equals methods in subclasses, no instance of this class should compare as equal to an instance of a subclass.

Parameters:

`obj` - object to be compared for equality

Returns:

`true` if the specified object is equal to this `DateFormatSymbols`

See Also:

`Object.equals(Object)`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.